

Arjun as a Boolean-Function Synthesis Engine

From Puura-simplified CNF to an AIG that defines every output

Mate Soos, Supratik Chakraborty, Kuldeep S. Meel

September 20, 2026

Synthesising Boolean functions $Y = f(X)$ from a relation $\exists Y. F(X, Y)$
— a counterexample-guided pipeline built on CryptoMiniSat & CaDiCaL.

The synthesis problem

Input: a CNF $F(X, Y)$ together with a partition of the variables — the user marks which are *inputs* X and which are *outputs* (to-define) Y .

Output: an **AIG** (And-Inverter Graph) that gives every $y_i \in Y$ as a Boolean function $y_i = f_i(X, y_{<i})$ such that $F(X, f(X))$ holds whenever $\exists Y. F(X, Y)$ does.

Where the formula comes from

Typical sources are **QBF** (the $\forall X \exists Y. F$ fragment used in Skolem-function synthesis [11]), **reactive synthesis** from temporal specs, and **circuit repair / don't-care optimisation** [12].

Arjun runs as a tool: `./arjun --synth input.cnf`. It first invokes **Puura** [10, 14] (CNF simplification — *not* covered here) and then attacks the *synthesis* problem on the simplified formula.

Three kinds of variables

Throughout the run Arjun classifies every CNF variable into one of three buckets

input vars in X — the free inputs. Their value is whatever the user / environment hands us; we never define them.

to-define vars in Y for which we still need to produce an AIG f_i . The set shrinks monotonically as the pipeline progresses.

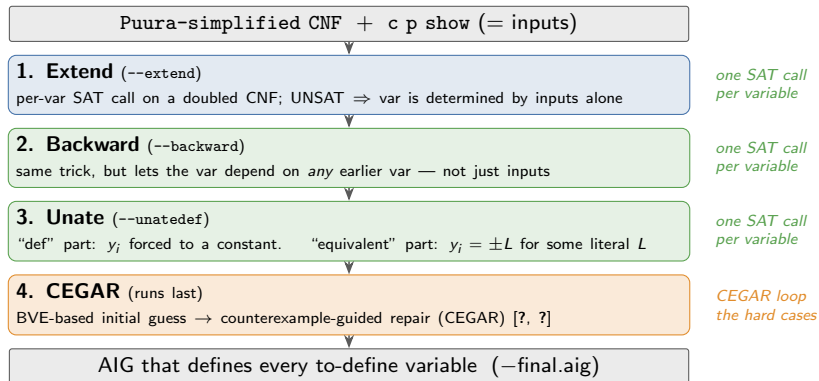
backward-defined vars $y_i \in Y$ that already have an AIG — “backward” in the sense that we proved y_i is determined by inputs and earlier outputs.

Invariant: every stage’s job is to move variables out of *to-define* and into *input* or *backward-defined*, with a verified AIG attached. When the to-define set is empty we are done.

Log header (bobtuint07neg)

Num input vars: 432 Num to-define vars: 1204 Num backward-synth-defined vars: 0

The synthesis pipeline



We try different CEGAR strategies, but we won't go into that here.

Stage 1 — Extend inputs via McMillan interpolant

Goal: prove that a single output y is fully determined by the *inputs alone* — and, if it is, extract the AIG defining it.

The construction (also used by [15], [13]):

- 1 Build the **doubled CNF** $F(X, Y_1) \wedge F(X, Y_2)$ — inputs X are shared, every non-input has two copies.
- 2 For each candidate y , ask the SAT solver: *can $y_1 \neq y_2$ while all inputs agree?* If **UNSAT**: y is a function of X .

From UNSAT to AIG. A CaDiCaL proof tracer reconstructs a **McMillan interpolant** over the input vars from the refutation proof. The interpolant *is* the AIG definition of y (idea: [3], [4]).

Log: extend on bobtuint07neg

```
[extend] Start unknown size: 353 ...[extend] Done.  True:  50 Unkn:  0
defined: 108 still to-define: 245
```

Stage 2 — Backward round: definitions over *anything*

Extend only accepts $y = f(X)$. **Backward** relaxes that: y_i may be defined by inputs *or* by other not-yet-defined y_j , as long as the dependency graph stays acyclic.

Same duplication trick, but the assumption set is *everything except* y_i . UNSAT means y_i is determined by all the other variables. The McMillan interpolant from CaDiCaL then gives the AIG — now possibly mentioning other y_j .

Why a separate pass? Extend is strictly cheaper (the interpolant only has to talk about X). Backward catches the “chained” definitions that Extend misses, at the cost of a longer-witness SAT problem.

Log: [backward-synth on bobtuint07neg](#)

```
[backward SYNTH] Start unknown size: 245 ... [backward SYNTH] Done.  TR: 146 UN: 0 FA: 99
defined: 99 still to-define: 146
```

Stage 3 — Unate definitions (two flavours)

Pure unate. For each remaining y , try the two SAT calls:

$F(X, Y) \wedge F(X, Y') \wedge (y = 0) \wedge (y' = 1)$ and the symmetric one. If either is UNSAT, then y is forced to a **constant** in every model that satisfies the formula on the “equal-on-others” miter. The constant becomes the AIG.

Equivalences When both flips are SAT, look at the two SAT witnesses M_1, M_2 . For each candidate literal L with $M_1[L] \neq M_2[L]$, run two more SAT calls. If both come back UNSAT we have proved $y = L$ or $y = \neg L$ is a valid definition.

Candidates are ordered: inputs first (cheapest, no cycle risk), then non-inputs that share a clause with y , then the rest.

Log: [unate_def](#) on [bobtuint07neg](#)

```
[unate_def] units:  1 eq defs:  0 eq calls:  1973 tested:  146
defined:  1 still to-define:  145
```

Stage 4 — The CEGAR core

Whatever Stages 1–3 leave is handed to the **CEGAR** stage: it starts with a *guess* for every remaining y_i and **repairs** it using counterexamples returned by a SAT solver.

Two SAT solvers, side by side.

- **cex solver** — **CaDiCaL** [5]. Holds the original CNF $F(X, Y)$ **and** a $y_i \mapsto \hat{y}_i$ copy of it (so we can ask: does our current candidate $\hat{y}(X)$ ever disagree with some real model?).
- **repair solver** — cached **CaDiCaL** used to extract an *unsat core* explaining *why* a particular candidate is wrong on a counterexample.

Initial guesses come from **CMSGen** [7] samples (100 by default in the BVE strategy) and from the BVE-style starting functions (next slide).

CEGAR's BVE starting guess

Before any repair, we need *some* candidate $\hat{y}_i(X, y_{<i})$ for every undefined y_i . The BVE strategy — the one we cover here — uses BVE-style resolution

- 1 Pick a polarity (the side with more occurrences) and walk all clauses where y_i appears with that sign.
- 2 For each such clause $(y_i \vee \ell_1 \vee \dots \vee \ell_k)$, form the conjunction $\neg \ell_1 \wedge \dots \wedge \neg \ell_k$ over *later* variables in the elimination order.
- 3 OR them together. Negate if we chose the negative side.

This is exactly the BVE definition: y_i is true iff at least one of its clauses needs it. The result is the **maximal** consistent guess: it is correct on every model where y_i is forced. Wherever it is wrong, the next SAT call will say so.

Log: BVE substitution

```
[cegar] BVE and substitute done.  funs:  145 funs/s:  8152.48 T: 0.02 mem:  17.68 MB
```

The repair loop

(1) Ask the cex solver: $F(X, Y) \wedge \neg F(X, \hat{Y}) \wedge \hat{y} = f(X)$ for current f .

SAT $\Rightarrow$ we have a counterexample $(X^*, Y^*, \hat{Y}^*)$ in which some $\hat{y}_i \neq y_i^*$. UNSAT $\Rightarrow$ done.

(2) Pick the *earliest* wrong y_r in the variable order.

(3) Ask the repair solver: *input-only* conflict first, then full conflict if needed.

Conflict C is the unsat core: a small set of literals s.t. $C \Rightarrow y_r = v_r$ where v_r is the correct value.

(4) Repair: $f_r \leftarrow (f_r \wedge \neg C) \vee (C \wedge v_r)$ — *compose-OR* / *compose-AND* on the AIG.

(5) Recompute $\hat{Y}$ on the ctx, recompute which variables still need repair, loop.

Each loop turn fires periodic stats log lines. Variable order is fixed up front by a BVE-derived ordering (minimises back-edges) so that “earliest wrong” is well-defined and avoids the cyclic-repair pitfall.

Craig interpolants as a smarter conflict

The conflict C from step (3) is one **corner** of the input space where the current f_r is wrong. The straightforward repair only excludes that corner.

Better: compute a **Craig interpolant** $I(X)$ [2, 3], *over inputs only*, separating the UNSAT proof's A side (this corner) from the B side (the formula). $I(X)$ generalises the conflict to the *entire* must-flip region of input space, so one compose-OR captures many repairs at once.

Implementation:

- A fresh CaDiCaL instance per repair, with a McMillan interpolation proof tracer attached.
- Walk the proof core (clauses reachable from the empty clause) and build the McMillan interpolant by hash-consed AIG composition.

Same Craig-interpolant idea drives Stages 2–3 above; only the partition (A, B) differs.

AIG rewriting and AIG-to-CNF – why they matter

The CEGAR hot loop is the SAT solver. Every repair grows the candidate AIG for some y_i ; every grown AIG must be Tseitin-encoded into the cex solver. If we encoded naively, the cex solver would balloon and dominate runtime.

AIG rewriting. Each repair round runs a simplification pass + an optional rewrite pass: constant propagation, complement elimination, idempotence, absorption, AND/OR distribution, XOR simplification, structural hash-consing, and **FRAIG-lite SAT-sweeping** [8].

Log: rewrite shrinks the AIG before encoding

```
[aig-rewrite] nodes: 13290 → 11354 (14.6% reduction) absorption: 263 distrib: 842 hash_hits: 190144
```

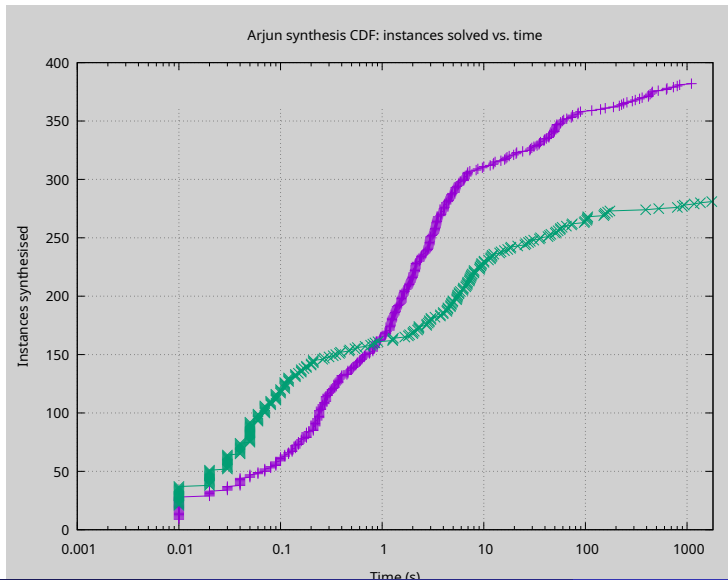
AIG-to-CNF encoding. K-ary AND/OR fusion, De Morgan flattening (free in the input-edge-neg model), ITE / MUX3 detection, and *cut-CNF* [9] encoding for small cones. One encoder per formula (cross-formula caching is unsound)

A worked example – bobtuint07neg

Stage	defined this stage	still to-define	wall time
initial (after Puura)	—	1204	—
synth-BVE	—	353	—
extend	108	245	0.04 s
backward	99	146	0.07 s
unate_def	1	145	1.02 s
CEGAR (BVE strategy)	145	0	4.62 s

- **Cheap stages first.** 209 of 353 vars defined before CEGAR starts — CEGAR only handles the hard 145.
- **CEGAR stats.** 1864 repairs in 1631 loops; 24% repair-solver cache-hit rate; conflict size avg 8.96 lits.
- **Time breakdown.** 39% in counterexample search, 21% in conflict extraction, 22% in recomputing candidate values — SAT calls and AIG bookkeeping dominate.

Results: Arjun vs Cadet



- **Synthesis is best done in stages** From cheap, global, non-iterative techniques (e.g. extend, backward) to expensive, iterative ones (CEGAR).
- Arjun synthesises Boolean functions on a Puura-simplified CNF: **input** / **to-define** / **backward-defined** are the key data structures
- **CEGAR** handles the residue: BVE-style initial guess, CMSGen samples, CryptoMiniSat-backed repair solver, conflict (or interpolant) generalisation, AIG composition repair.
- **AIG rewriting + AIG-to-CNF** are key elements, they keep the cex solver small so the CEGAR loop stays fast.
- **Available for every major OS**: Linux, Mac, Windows, compiled for x86, ARM, and WASM.

References I

Mark H. Liffiton, Karem A. Sakallah. *Searching for Autarkies to Trim Unsatisfiable Clause Sets*. SAT 2008.

William Craig. *Three uses of the Herbrand–Gentzen theorem in relating model theory and proof theory*. J. Symbolic Logic, 1957.

Kenneth L. McMillan. *Interpolation and SAT-based model checking*. CAV 2003.

Jie-Hong Roland Jiang, Hsuan-Po Lin, Wei-Lun Hung. *Interpolating Functions from Large Boolean Relations*. ICCAD 2009.

Armin Biere. *CaDiCaL at the SAT Race 2019*. SAT Race 2019.

Mate Soos, Karsten Nohl, Claude Castelluccia. *Extending SAT Solvers to Cryptographic Problems (CryptoMiniSat)*. SAT 2009.

Priyanka Golia, Mate Soos, Sourav Chakraborty, Kuldeep S. Meel. *Designing Samplers is Easy: The Boon of Testers (CMSGen)*. FMCAD 2021.

Alan Mishchenko, Satrajit Chatterjee, Roland Jiang, Robert K. Brayton. *FRAIGs: A Unifying Representation for Logic Synthesis and Verification*. ERL Technical Report, UC Berkeley, 2005.

Niklas Eén, Alan Mishchenko, Niklas Sörensson. *Applying Logic Synthesis for Speeding Up SAT*. SAT 2007.

References II

Tuukka Korhonen, Matti Järvisalo. *Integrating Tree Decompositions into Decision Heuristics of Propositional Model Counters (SharpSAT-td)*. CP 2021.

Aina Niemetz, Mathias Preiner, Florian Lonsing, Martina Seidl, Armin Biere. *Resolution-Based Certificate Extraction for QBF (DepQBF)*. SAT 2012.

Ajith K. John, Shetal Shah, Supratik Chakraborty, Ashutosh Trivedi, S. Akshay. *Skolem Functions for Factored Formulas*. FMCAD 2015.

Arijit Shaw, Kuldeep S. Meel. *Counting on Definability: Tools and Tactics for Projected Model Counting*. Recent work building on the Arjun definability ideas.

Mate Soos, Kuldeep S. Meel. *BIRD: Engineering an Efficient CNF-XOR SAT Solver and Its Applications to Approximate Model Counting*. AAAI 2019.

Mate Soos, Stephan Gocht, Kuldeep S. Meel. *Tinted, Detached, and Lazy CNF-XOR Solving and Its Applications to Counting and Sampling*. CAV 2020.